ELI — Orchestration And Agents

ELI v2.5.10

October 2026

Contents

ELI Orchestration & Agents — Full Topology **1**
Two agent stacks . . . 1
Planning artifacts . . . 4
Where it is still weak . . . 5
Habits reach the chat path . . . 5
Custom agents have a specification and a trust chain . . . 5

ELI Orchestration & Agents — Full Topology

> **Updated 2026-09-30.** All CHAT modes run the orchestrator at scaled depth; retrieval is unified in `eli/memory/retrieval.py`; stage 12 learning is centralised in `learning_coordinator.py`; the direct-versus-synthesise gate (`_deterministic_direct_payload_actions`) has been audited against what the executor handlers actually return. Agent-bus confidence now has a relevance term; MULTI_COMMAND gained real dependency awareness and crash-survival.

Read-only reference; nothing here changes behaviour.

Source files: - `eli/cognition/orchestrator.py`: the orchestrator (12-stage pipeline) - `eli/cognition/agent_bus.py`: the parallel 15-agent specialist bus - `eli/memory/retrieval.py`: shared turn retrieval (bus and orchestrator) - `eli/cognition/learning_coordinator.py`: stage 12 `finalize_turn()` - `eli/kernel/pipeline_trace.py`: canonical S01–S12 logging - `eli/kernel/engine.py`: wiring and the dispatch gate - `eli/execution/execution_planner.py` the typed plan model

Two agent stacks

ELI has **one primary cognition path** for CHAT and a **parallel specialist bus** that the orchestrator composes (or that the engine falls back to).

1. `AgentOrchestrator` (`orchestrator.py`)

The 12-stage cognitive pipeline. The engine calls it for every CHAT mode, Quick through Expert, at a depth chosen by `mode_orchestrator_depth()` and `orchestrator_planner_mode()` (`reasoning_modes.py`). Components:

- **`PlannerAgent.plan_retrieval()`** produces a mode-aware retrieval plan:
  - `fast`: keyword only, no FAISS, document passages only when the question is about a document, knowledge graph only for identity, one ReAct iteration, no HyDE;
  - `balanced` (default): keyword, semantic and knowledge graph, document passages (the best ones when the question is about a document, otherwise only passages that clearly bear on it), three ReAct iterations;

  - `deep`: everything, large budgets, full HyDE, three ReAct iterations. The orchestrator then attaches the time window found in the question (`query_planner.parse_window`) to the plan.
- **`OrchestratorMemoryAgent`** delegates to `retrieve_for_turn()` in `memory/retrieval.py` (shared with the bus): HyDE expansion, keyword and FTS5, FAISS semantic and knowledge graph, `hybrid_merge`, then a heuristic rerank (`rerank_candidates`). Document passages come from `engine.document_rag` (`memory/document_index.py`) and are not merged into the evidence rows: they are shown whole, best first, in their own block. Sequential by design: the llama.cpp embedder is not thread-safe.
- **`ExecutorAgent`** is a thin wrapper over `executor_enhanced.execute`.

Flow inside `AgentOrchestrator.run()`:

- **Non-CHAT actions**: dispatch the AgentBus for specialist evidence, then a **ReAct observation loop**: run the executor, ask the loaded model for `ANSWER` or `TOOL:<action> <args>`, chain to the next tool and accumulate observations. One iteration in fast mode, up to three otherwise. The proposed tool is validated against the executor's `SUPPORTED_ACTIONS` (207 actions) and the loop stops on an unknown action; `intent["args"]` are merged rather than overwritten. For "grounded synthesis" actions the observations are assembled into context and passed to the model; for direct actions the executor result is returned as is.
- **CHAT**: planner → shared retrieval → `dispatch_specialists()` (mode-aware fan-out; memory skipped when already prefetched) → context assembly and a retrieval-diagnostics block → persona handoff → generation. The assembled context is ordered by what must survive a trim (`context_budget._BLOCK_PRIORITY`): retrieval diagnostics, the turn record (only when the user asks about ELI's own behaviour), the period log, document passages, verified memories, reranked evidence, recent turns. Private reasoning modes (Normal, Advanced, Research, Expert) hand off to `engine._run_chat_reasoning_loop`. The bus is composed on the CHAT path and is not bypassed in Quick mode.

Following up on what ELI said. Whatever a reply offers, asks or proposes, the user's next message is read against it, in code, whatever the conversation is about (`runtime/pending_proposal.py`).

What a reply leaves open. `_store_assistant_turn` calls `read_reply` on every reply and stores the result, replacing the last one (it lapses after 30 minutes):

- **steps** ELI offered, each with the number it was listed under. A step is an *action* when the router has a rule for its words; a *task* when ELI would do it by writing (explain, draft, compare, plan); *explicit* when it is something only the user's own words may start (`llm_intent._EXPLICIT_ONLY_ACTIONS`). An action is kept wherever ELI names it. A task is kept only where ELI asked for a go-ahead: its own offer, as a question ("Want me to walk you through it?") or as a statement ("let me know if you'd like me to draft it", "I can turn this into a checklist if you'd like"), or the items of a list its question points at ("Shall I go ahead with any of these?"). A list of facts followed by an unrelated offer is not a set of tasks, and "can I help with anything else?" offers nothing to carry out.
- the **question** the reply ended on, if it ended on one.
- a **detail an action asked for**. A handler that returns `awaiting` ({command, action, needs}) or `choices` has it stored by the executor ("When is it?", "Which should go?", "Where?"). `engine._asks_the_user` keeps such a result out of the failure log and the re-plan, and shows its text as it is; so is any not-ok result whose text ends by asking.

Offers are read from the reply as the user saw it. The stored copy has closing filler removed by the output governor, which used to delete "let me know if you'd like me to …" along with "let me know if you need anything else"; a particular offer is now kept.

What the next message does (`router._stage_pending_proposal_confirm`):

- a decline drops what was offered;
- consent (`is_consent`: "yes please", "go ahead", "sounds good", "do 1-3") covers the steps picked by number or by name, else all of them (`chosen_items`);
- a pick needs no "yes" (`selection`: "the second one", "2 and 3", "the abstract one");
- the detail an action asked for (`is_the_detail`: a day or time, or a place) runs the original request again with it;

- anything else is a new message. One word is then an answer, not a fragment to ask again about.

Carrying it out (`engine._carry_out_agreed`): agreed actions run through the executor (`MULTI_COMMAND` with `results_only`, so the reply is each step's outcome). Agreed tasks run as a nested turn whose message is the task in the user's voice (`task_message`), so retrieval, budget and the reply are about the task and not about the words "yes please"; that turn is CHAT without routing or resolving ELI's words again, and `engine._follow_up_line` tells the model the user agreed and what has already been done. An explicit-only step is never run and never handed to the model: the reply says which words to use. The outer turn stores what the user typed and the reply they saw.

A short reply after a question that is not a pick or a detail goes to the model with the question stated (`engine._follow_up_line`: "your last message ended by asking …; their message is the answer; act on it").

Three older rules still hold. *One routing per turn*: `engine._route_once` caches the router's answer, because routing has an effect (a "yes" spends the offer it confirms). *ELI's own sentence is not a command*: `_stream_with_followthrough` runs a promise in a reply only when it is a read (`_FOLLOWTHROUGH_READS`) or something the user's message asked for. *A mention with a clock time* that is not on the calendar gets a fixed offer sentence (`_agenda_offer`); a part of the day ("tonight") is not offered back as a time.

DAG pool tasks run inside a copy of the caller's context (`core/dag.py`), so an action executed on a worker thread is recorded on the turn and carries the user id.

The direct-versus-synthesise decision is a set membership test in `eli/kernel/engine.py`:

- `_DIRECT_RESULT_ACTIONS` (194 entries, module level; the control path reads it as `_deterministic_direct_payload_actions`): the executor's `content` or `response` is returned verbatim in quick mode; in other modes such an action may be re-narrated by `_compact_grounded_synthesis()` (constrained to quote from evidence, validated against it, falling back to the raw evidence).
- `_verbatim_always_actions` (16 entries): verbatim in every mode.
- `_shown_as_is(action, reasoning_mode)` is the same decision for the two other places an action result is returned (the agent-bus path and the general executor path). They used to keep shorter lists of their own, so a calendar or timer result went through a full generation there and came back reworded. `_ALWAYS_AS_IS` (calendar, alarm, timer, `MULTI_COMMAND`, `SEQUENCE`) is verbatim in every mode on those paths.
- `eli/runtime/response_contracts.py::_QUICK_ACTIONS` (8 entries) only feeds the prompt header and never decides verbatim versus synthesis.

The 187 routable actions were each checked against their executor handler's real return shape (not assumed from the name): raw file reads, shell output, MCP results, transcriptions and OCR text, confirmations and status reports are verbatim; `ANALYZE_IMAGE`, `ANALYZE_PDF[_FOLDER]`, `SCREEN_READ_ANALYZE`, `DATA_FABRICATOR`, `GENERATE_PROJECT`, `SEQUENCE` and `MULTI_COMMAND` already return finished text and are verbatim too. Excluded on purpose, each with a reason: `FIX_FILE` and `GENERATE_SCRIPT` (their `content` is a JSON event for the GUI), `RUN_TESTS` (meant to be summarised), `CHAT` (the model call itself), `SHOW_DIFF` (routes to `chat()`), `WEB_SEARCH` (results are evidence, not the answer), `CODE_SOLVE` (generative, goes to the coding agent), `EXECUTE_GOAL` and `NOOP` (no single handler to verify). `tests/test_deterministic_actions_cover_status_and_read_file.py` holds the audited lists and a completeness test that fails if a routable action lands in neither set nor the exclusion list.

2. `AgentBus`: the parallel 15-agent fan-out (`agent_bus.py`)

15 agents in `_ALL_AGENTS`, each a `_BaseAgent` subclass with a `name` and `timeout_s`:

#	name	timeout_s	accesses
1	`memory`	5.0	SQLite, FTS5, FAISS; self-gates
2	`system`	8.0	direct action execution (`SYSTEM_ACTIONS`)
3	`habit`	3.0	`user_patterns`, learned and detected habits
4	`self_improvement`	3.0	failures, improvement proposals, corrections

#	name	timeout_s	accesses
5	proactive	3.0	suggestion and anticipation
6	frontier	5.0	frontier and awareness model
7	plugin	6.0	plugin registry (PLUGIN_ACTIONS)
8	capability	6.0	capability manifest
9	voice	5.0	TTS and voice subsystem
10	orchestrator	3.0	bus-level planner; emits a plan dict only
11	file_code	4.0	source tree and code introspection
12	reflection	4.0	reflection log and insights
13	introspection	4.0	runtime and cognition self-inspection
14	knowledge_graph	3.0	entity and relation graph (runs after memory)
15	critic	2.0	verifies retriever output (runs after memory, file_code, knowledge_graph, system)

SpecAgent instances (user-defined specifications) and custom agents register on top of these.

Execution (AgentBus.dispatch):

- **Selective fan-out**: tiny filler chat → {memory, orchestrator}; non-chat → _select_agents_for_intent (a minimal set from a keyword and action ladder); plain CHAT → broad fan-out across all enabled agents.
- **Dependency DAG**: _AGENT_DEPENDENCIES orders agents in topological layers, running the agents of a layer in parallel; knowledge_graph waits for memory, and critic waits for its four retrievers. If the DAG fails the bus falls back to a flat parallel run.
- **Per-agent hard timeout**: a timeout or exception becomes a failed AgentResult and never blocks the response.
- **Aggregation** (_aggregate_confidence): per-agent contribution = evidence quality × evidence density × a calibration learned per (agent, action) × relevance (2026-09-29: term_overlap(user_input, representative_text) → relevance_gate(), gated on ≥40 chars of text; a fluent, evidence-dense result that's off-topic for what was actually asked no longer scores as highly as one that's on-topic — env ELI_AGENT_BUS_RELEVANCE_GATE to disable); a single-agent cap; an empty-bus ceiling; a corroboration bonus when several agents contribute. CriticAgent's corroboration score gets the same relevance discount: two sources can agree strongly with each other while both being off-topic.

When each runs

Situation	Path
CHAT (Quick, Normal, Advanced, Research, Expert)	**AgentOrchestrator** at mode depth → shared retrieval → dispatch_specialists()
Non-CHAT action (any mode)	**AgentOrchestrator** → bus and ReAct loop
Orchestrator returns None or raises	falls back to **AgentBus.dispatch()** directly
Phatic or ultra-short filler (engine heuristic)	may use a lean bus profile inside the orchestrator

Stage 12 side effects (store the turn, publish meta, _learn_from_result) run through learning_coordinator.finalize_turn(), one entry point for every CHAT exit. Exits that never reach it (the orchestrator stream, middleware answers) are completed by engine._finalize_turn_backstop, which runs once when a top-level turn ends normally and does only what the turn did not: the user's message, then the reply, episodic memory for chat-like turns, and the response meta. Nested turns (a multi-question split, a follow-through re-run) store nothing; their parent stores the real message and the reply the user saw.

Planning artifacts

1. **ReAct loop**: the only planner that sequences execution (tool → observe → decide → next).

2. **`PlannerAgent.plan_retrieval`**: plans retrieval budgets, not actions; used every CHAT turn.
3. **Bus `OrchestratorAgent` plan**: an `orchestrator_plan` dict stored in the trace for display, persona handoff and status; never executed.
4. **`execution_planner.build_execution_plan`** (`ExecutionPlan`, `PlanStep`): the canonical typed plan. `AgentBus.dispatch` builds it each turn, injecting the result of `_select_agents_for_intent` as `agent_profile`, and drives the active agents through it; it is exposed as `DispatchResult.execution_plan`. `EXECUTE_GOAL` also builds a plan through it. **`.steps` is never executed anywhere** — only `.agent_profile` (an agent-name filter) is read; `EXECUTE_GOAL` renders `.steps` as decorative numbered text, nothing more. Confirmed dead scaffolding for real step execution, not a mechanism to extend.
5. **`MULTI_COMMAND` (`command_dependency_graph.py`, 2026-09-29)**: the only mechanism in the codebase that actually, mechanically executes a multi-part request (as opposed to items 3-4 above). One optional LLM call identifies real dependency edges among an already-split command list (validated through the shared `eli.core.dag` engine, falling back to fully independent on any failure); the executor runs the commands in topological layers, and a step whose dependency failed is skipped and reported as such rather than blindly run. Crash-survival is separate (`command_sequence_log.py`): each step's outcome is durably recorded, and a resubmit of the same text within an hour skips steps already known to have succeeded.

Where it is still weak

1. **Bus agents cannot consume each other's output** beyond the declared DAG edges; only `knowledge_graph` and `critic` have dependencies.
2. **Planning is only partly consolidated.** The engine's `_build_runtime_orchestrator_plan` (a rich stage dict) and the bus `OrchestratorAgent` plan coexist with the typed `ExecutionPlan`; only the ReAct loop executes a sequence.
3. **Timeouts do not cancel work.** `future.result(timeout)` only stops waiting; a timed-out write-capable agent (memory, habit) can still land a late database write.
4. **Confidence is coupled to a fixed evidence-key schema.** `_evidence_density` counts known keys; a custom agent returning useful prose under an unknown key contributes zero.
5. **No early exit for direct actions; failures are debug-only.** The bus waits for the slowest selected agent even when a direct `action_result` exists, and timeouts log at debug level with no health surface, although an `agent_metrics` table exists.

Fixed and worth knowing: `_apply_runtime_policy_timeouts()` is applied to built-ins and again after `_load_custom_agents()`, so custom agents get hardware-adapted timeouts too.

Highest leverage now: an agent-health surface built from `agent_metrics` and `pipeline_trace`, and a cooperative cancel token for write-capable agents.

Habits reach the chat path

The bus `HabitAgent` reads both `get_habit_rules()` and `get_detected_habits()` (the `habits` table the proactive daemon fills), emits a `summary` and `detected_habits`, and `DispatchResult.to_context_block` renders the summary into the chat context. Goal autogenesis (`planning/goal_autogenesis.py`) feeds the autonomy and goal-tick stack from ELI's own signals; see `runtime_planning_world.md`.

Custom agents have a specification and a trust chain

A custom agent was once a `.py` file dropped in a directory with a `name`, a `timeout_s` and an optional free-text persona. It is now **data, not code**.

`AgentSpec` (`eli/cognition/agent_spec.py`)

Field	Required	Purpose
`objective`	yes	one sentence on what this agent is responsible for
`system_prompt`	yes	the instruction the model receives
`triggers`	at least 1	`keyword`, `regex`, `action`, `always`
`success_criteria`	at least 1	runnable checks: contains, not_contains, regex, min/max length, non_empty, is_json
`examples`	no	input and expected checks, so the agent can be tested before going live
`permissions`	no	capabilities from the plugin vocabulary, gated at run time

`validate()` refuses vagueness: an objective under 25 characters or matching a placeholder list (`todo`, `does stuff`, `helper`, ...) is rejected, as is a system prompt under 40 characters. An agent with no trigger is refused because it would never run; one with no success criterion is refused because nothing could tell whether it worked; an `always` trigger is allowed with a warning about latency. `evaluate(output)` runs the criteria and returns a score, which is the measure the wizard's test step uses and the confidence `SpecAgent` reports. `content_hash()` excludes `created` and `enabled`, so re-saving a spec does not invalidate a trust grant while a real edit does.

`SpecAgent (agent_bus.py)`

Runs a spec: check triggers, check declared permissions, call the local model with the spec's system prompt, score the output against the criteria. An agent that fails its own success test contributes nothing. Because a spec executes no arbitrary code, spec agents need no trust grant; the hash, scan and provenance chain applies only to code agents.

Loader

- `_custom_agent_dirs()` puts the data directory first (`<data>/agents/custom`), so a created agent has somewhere valid to live on any install.
- Trust is checked through `agent_trust.inspect()` (see `security.md`), which reports why an agent was not loaded.
- `agent_load_report()` records the per-agent outcome so the GUI can show the reason.
- `reload_custom_agents()` re-scans specs and code without a restart and replaces previously registered spec agents.